

Software Requirements Specification

for

AI Smart Assistant

Prepared by Team **SENTINELS**

Department of Computer Engineering,
University of Peradeniya

February 20, 2026

Contents

1	Introduction	1
1.1	Purpose	1
1.2	Document Conventions	1
1.3	Intended Audience and Reading Suggestions	1
1.4	Product Scope	1
1.5	References	1
2	Overall Description	2
2.1	Product Perspective	2
2.2	Product Functions	2
2.3	User Classes and Characteristics	2
2.4	Operating Environment	2
2.5	Design and Implementation Constraints	3
2.6	User Documentation	3
2.7	Assumptions and Dependencies	3
3	External Interface Requirements	3
3.1	User Interfaces	3
3.2	Hardware Interfaces	3
3.3	Software Interfaces	4
3.4	Communication Interfaces	4
4	System Features	4
4.1	Intelligent Screen Capture	4
4.1.1	Description and Priority	4
4.1.2	Stimulus/Response Sequences	4
4.1.3	Functional Requirements	4
4.2	OCR Text Extraction	4
4.2.1	Description and Priority	4
4.2.2	Stimulus/Response Sequences	5
4.2.3	Functional Requirements	5
4.3	Context-Aware AI Analysis	5
4.3.1	Description and Priority	5
4.3.2	Stimulus/Response Sequences	5
4.3.3	Functional Requirements	5
4.4	Action Automation	5
4.4.1	Description and Priority	5
4.4.2	Stimulus/Response Sequences	5
4.4.3	Functional Requirements	6
5	Other Nonfunctional Requirements	6
5.1	Performance Requirements	6
5.2	Safety Requirements	6

5.3	Security Requirements	6
5.4	Software Quality Attributes	6
5.5	Business Rules	7
6	Other Requirements	7
A	Glossary	7
B	Analysis Models	7
C	To Be Determined List	7

1 Introduction

1.1 Purpose

The purpose of this document is to specify the software requirements for the AI Smart Assistant System, Version 1.0. This SRS describes the scope, functional requirements, and nonfunctional requirements for the system, which is designed to assist users by interpreting screen content and providing automated guidance.

1.2 Document Conventions

This document follows the IEEE 830-1998 standard for Software Requirements Specifications. All requirements follow the "Shall" convention for mandatory actions. Identifiers are structured as REQ-[Numerical ID]. Typographical conventions include **bold face** for UI elements and *italics* for emphasis.

1.3 Intended Audience and Reading Suggestions

This document is prepared for stakeholders, developers, and project evaluators. Reading suggestions:

- **Developers & Designers:** Focus on Section 3 (Interfaces) and Section 4 (Features).
- **Project Supervisors:** Review Section 1 (Scope) and Section 2 (Overall Description).
- **Testers:** Refer primarily to Sections 4 and 5 for verifiable requirements.

1.4 Product Scope

The AI Smart Assistant is a desktop application aimed at bridging the gap between complex software interfaces and user tasks. By utilizing Optical Character Recognition (OCR) and Large Language Models (LLMs), the system provides context-aware help, error troubleshooting, and automation of repetitive tasks.

1.5 References

1. IEEE Std 830-1998, IEEE Recommended Practice for Software Requirements Specifications.
2. Tesseract OCR Documentation (v5.0).
3. OpenAI / Local LLM (Llama/Mistral) API Documentation.

2 Overall Description

2.1 Product Perspective

The AI Smart Assistant is a new, self-contained desktop product designed for modern operating systems. It functions as a thin layer above the OS, monitoring the display buffer to provide intelligent feedback without requiring deep integration into existing third-party applications.

2.2 Product Functions

The system's core capabilities include:

- **Visual Intelligence:** Capturing and identifying objects/text on the user's screen.
- **Context Analysis:** Mapping screen data to a knowledge base of software tutorials and error codes.
- **Active Guidance:** Displaying real-time overlays with instructions.
- **Action Automation:** Simulating peripheral inputs to execute multi-step solutions.

2.3 User Classes and Characteristics

- **Novice Users:** Individuals unfamiliar with specific software tools who require step-by-step hand-holding.
- **Power Users:** Users looking to speed up workflows through intelligent automation and shortcuts.
- **System Administrators:** Users needing to troubleshoot complex system errors via AI-driven diagnostics.

2.4 Operating Environment

The software shall operate under the following environment:

- **Operating System:** Windows 10/11
- **Hardware:** Minimum 12GB RAM recommended for local AI execution; NVIDIA GPU with 4GB+ VRAM for optimal performance.(as for MVP since we are not using cloud host for The AI)
- **Dependencies:** Python 3.10+, CUDA Toolkit (for local acceleration),LangChain, LlamaIndex

2.5 Design and Implementation Constraints

1. **Hardware Constraints:** Must run efficiently on consumer-grade hardware; local AI models must be optimized (e.g., GGUF/AWQ quantization).
2. **Privacy Constraints:** All OCR data must be processed locally unless cloud processing is explicitly enabled.
3. **Performance Constraints:** OCR latency must not exceed 500ms for full-screen capture.

2.6 User Documentation

Comprehensive user documentation will be delivered as an integrated help system and a printable PDF Installation and Troubleshooting Guide.

2.7 Assumptions and Dependencies

- **Permissions:** The system assumes the user provides Accessibility and Screen Recording permissions.
- **Internet Connectivity:** Optional, but certain high-fidelity AI features may depend on external API availability.

3 External Interface Requirements

3.1 User Interfaces

The AI Smart Assistant shall provide a semi-transparent, non-intrusive overlay panel. This interface includes:

- A chat box for direct interaction with the AI.
- Contextual action buttons (e.g., “Fix This Error”, “Show Tutorial”).
- Highlighted screen regions indicating areas of interest detected by OCR.

3.2 Hardware Interfaces

- **Input Devices:** The system shall monitor keyboard events for global hotkeys and mouse events for coordinate-based automation.
- **Screen Capture:** Interfaces with the GPU/Display buffer to retrieve frame data for processing.

3.3 Software Interfaces

- **OCR Library:** Integration with Tesseract or EasyOCR through Python subprocess or direct API calls.
- **LLM Backend:** Connection to a local Inference Server (Ollama/vLLM) or OpenAI API via RESTful JSON endpoints.
- **System APIs:** Windows Win32 API or X11/Wayland protocols for screen capture and input simulation.

3.4 Communication Interfaces

The system communicates with external AI services over HTTPS using TLS 1.2+ for data security. Local inter-process communication (IPC) between the UI and AI backend is handled via WebSockets or specialized message queues.

4 System Features

4.1 Intelligent Screen Capture

4.1.1 Description and Priority

Allows the user to select a portion of the screen or a specific window for analysis. Priority: High.

4.1.2 Stimulus/Response Sequences

1. User triggers hotkey (**Ctrl + Alt + Shift+O**).
2. System dims the screen and enables a crosshair selection tool.
3. User selects a rectangular region.
4. System saves the region to memory and initiates OCR.

4.1.3 Functional Requirements

- **REQ-SC-1:** The system shall identify the active window coordinates automatically.
- **REQ-SC-2:** The system shall support multi-monitor setups for selection.

4.2 OCR Text Extraction

4.2.1 Description and Priority

Converts the visual data from captured images into machine-readable text. Priority: High.

4.2.2 Stimulus/Response Sequences

1. Image data is sent to the OCR module.
2. OCR module applies preprocessing (grayscale, noise reduction).
3. OCR returns a structured text object with confidence scores.

4.2.3 Functional Requirements

- **REQ-OCR-1:** The system shall achieve >90% accuracy on standard English fonts at 10pt size.
- **REQ-OCR-2:** The system shall preserve indentation and basic layout from the source screen.

4.3 Context-Aware AI Analysis

4.3.1 Description and Priority

Interprets the extracted text to provide solutions. Priority: High.

4.3.2 Stimulus/Response Sequences

1. OCR text is bundled with a system prompt and sent to the LLM.
2. AI generates a concise solution or step-by-step guide.
3. The suggestion is rendered in the UI overlay.

4.3.3 Functional Requirements

- **REQ-AI-1:** The AI shall format responses in Markdown for readability.
- **REQ-AI-2:** The AI shall provide a "Simulate" option if the task is automatable.

4.4 Action Automation

4.4.1 Description and Priority

Executes tasks on the user's behalf through input simulation. Priority: Medium.

4.4.2 Stimulus/Response Sequences

1. User clicks "Automate" on a suggestion.
2. System shows a preview of actions.
3. User confirms action.
4. System executes keyboard/mouse sequences.

4.4.3 Functional Requirements

- **REQ-AUT-1:** System shall provide an "Emergency Stop" hotkey during automation.
- **REQ-AUT-2:** System shall verify the state of the screen before each simulated click.

5 Other Nonfunctional Requirements

5.1 Performance Requirements

The system shall adhere to the following performance benchmarks:

- **Throughput:** Screen capture and OCR processing shall complete within 1.5 seconds on the recommended hardware.
- **Inference Speed:** AI response generation for standard troubleshooting tasks shall not exceed 3.0 seconds per request.

5.2 Safety Requirements

- **Emergency Stop:** A global hotkey (e.g., **Shift + Esc**) shall be active during any automated simulation to immediately cease all mouse and keyboard actions.
- **User Confirmation:** No administrative or destructive action shall be performed without a blocking modal confirmation dialog.

5.3 Security Requirements

- **Data Privacy:** All captured images and extracted text shall be stored in temporary RAM and cleared immediately after processing.
- **Consent:** External API calls for cloud AI shall only be made if the user has explicitly checked the "Enable Cloud Features" option.

5.4 Software Quality Attributes

- **Availability:** The system shall be available for use within 5 seconds of application launch.
- **Robustness:** The system shall gracefully handle low-confidence OCR results by requesting user clarification instead of proceeding with potentially incorrect data.
- **Testability:** Every functional requirement (REQ-ID) shall be mapped to a corresponding unit or integration test case.

5.5 Business Rules

Automated simulation features are restricted to “Pro” or “Authenticated” user sessions to prevent unauthorized script execution on sensitive systems.

6 Other Requirements

The architecture shall allow for future support of alternative input methods, such as voice commands and eye-tracking interfaces, through a modular plugin system.

A Glossary

OCR *Optical Character Recognition* — The mechanical or electronic conversion of images of typed, handwritten, or printed text into machine-encoded text.

AI *Artificial Intelligence* — The simulation of human intelligence processes by machines, especially computer systems.

LLM *Large Language Model* — A type of AI model designed to understand and generate human-like text.

GGUF *GPT-Generated Unified Format* — A file format for storing models for inference.

B Analysis Models

The following models are pending final design approval:

- **Architecture Diagram:** Shows the decoupling of the UI, Analysis Engine, and Automation Module.
- **Data Flow Diagram:** Traces pixels from capture to AI response.

C To Be Determined List

1. Final list of supported third-party cloud AI vendors.
2. Finalized safety policy for file-system-level automation.